

Microsoft

(AI-300)

Operationalizing Machine Learning and Generative AI Solutions

Total: **108 Questions**

Link: <https://examauthor.com/papers/microsoft/ai-300>

Question: 1

Case Study -

This is a case study. Case studies are not timed separately from other exam sections. You can use as much exam time as you would like to complete each case study. However, there might be additional case studies or other exam sections. Manage your time to ensure that you can complete all the exam sections in the time provided. Pay attention to the Exam Progress at the top of the screen so you have sufficient time to complete any exam sections that follow this case study.

To answer the case study questions, you will need to reference information that is provided in the case. Case studies and associated questions might contain exhibits or other resources that provide more information about the scenario described in the case. Information provided in an individual question does not apply to the other questions in the case study.

A Review Screen will appear at the end of this case study. From the Review Screen, you can review and change your answers before you move to the next exam section. After you leave this case study, you will NOT be able to return to it.

To start the case study -

To display the first question in this case study, select the "Next" button. To the left of the question, a menu provides links to information such as business requirements, the existing environment, and problem statements. Please read through all this information before answering any questions. When you are ready to answer a question, select the "Question" button to return to the question.

Background -

Fabrikam Inc. is a mid-sized healthcare analytics company that provides population health dashboards and predictive insights to regional hospital systems across the United States. Fabrikam Inc. customers rely on near real time analytics to monitor patient flow, staffing needs, and readmission risks. They use multiple traditional forecasting machine learning models for predictions.

Fabrikam Inc. has an established Microsoft Azure footprint. The company uses Jupyter Notebooks that run on a local server as the primary development environment. The data science team is experiencing scalability, asset management and code management issues with the current development platform. Fabrikam Inc. plans to migrate to a cloud-based development environment to mitigate the issues.

Additionally, the company plans to implement a Retrieval-Augmented Generation (RAG)-based chat application for client support. Leadership requires the application to be developed and deployed with a low operational risk.

Current Environment -

Fabrikam Inc. operates a single Azure subscription that has the following components:

- Azure Data Lake Storage Gen2 that contains de-identified clinical and operational datasets

- Azure AI Search indexing curated analytical documents and reference materials

- A small set of Python-based training scripts maintained by data scientists

- Azure OpenAI Service with deployed foundational models

- A Microsoft Foundry resource for building a RAG-based solution

- Evaluation data has manually defined expected responses.

The current challenges faced by the data science team include the following:

- Model training jobs are run manually from notebooks.

- Experiment tracking is inconsistent

- Model versions are registered without standardized metadata.

- Deployment is performed manually by data scientists, with limited rollback capability.

- The team has no standardized evaluation process for generative AI outputs.

- The environment currently allows public network access. Authentication relies on user accounts rather than managed identities. Compute targets are manually created and shared across experiments. This has led to resource contention during peak usage.

Business Requirements -

Fabrikam Inc. has the following business requirements for the modernization initiative:

- Provide a conversational interface that answers analytics questions by using internal documents and datasets.

- Ensure that sensitive healthcare-related data is not exposed outside the Fabrikam Inc. Azure tenant.

- Enable repeatable and auditable model training and deployment processes.

- Support experimentation to compare prompt strategies and fine-tuned models.

- Align the model with the ranked preferences and optimize behavior for the long term.

- Minimize disruption to existing analytics workloads during rollout.

Technical Requirements -

To support the business goals, Fabrikam Inc. identifies these technical requirements:

- Use Azure Machine Learning workspaces to centrally manage data assets, models, and environments.

- Implement experiment tracking and model versioning for all training jobs.

- Orchestrate training and evaluation by using pipelines rather than manually running notebooks.

- Deploy traditional machine learning models with support for staged rollout and rollback.

- Improve RAG-based solution output quality.

- Use the existing evaluation datasets that are based on real data with input-output pairs.

- Apply advanced fine-tuning techniques only when prompt engineering is insufficient

Issues and Constraints -

Fabrikam Inc. must comply with internal security policies that require the company to restrict network access and avoid long-lived secrets. The data science team has limited Azure DevOps experience, so solutions must favor managed services and automation over custom infrastructure.

Cost predictability is important. Leadership prefers serverless or managed compute options where possible but is willing to approve dedicated compute for stable production workloads.

Problem Statement -

Fabrikam Inc. must design and implement an Azure-based AI operations solution that enables reliable training, evaluation, deployment, and iteration of generative AI models. The solution must support experimentation and gradual rollout while ensuring governance, security, and operational stability. The data science and platform teams must collaborate to deliver this solution by using Azure Machine Learning and Microsoft Foundry capabilities.

You need to standardize how Fabrikam Inc. manages machine learning assets.

Which action should you perform first?

- A. Register assets in the Azure Machine Learning registry.
- B. Create a shared Azure Machine Learning workspace.
- C. Deploy a managed online endpoint.
- D. Create a new Microsoft Foundry project.

Answer: B

Explanation:

Technical Justification for the Correct Option

Correct Answer: B. Create a shared Azure Machine Learning workspace.

Why B is the best option: Before managing machine learning assets effectively, a centralized, unified environment is necessary. Creating a shared Azure Machine Learning workspace provides a single source of truth for the data science team to collaborate on, manage, and track experiments, models, datasets, and deployments. This aligns with the business requirement to "Provide repeatable and auditable model training and deployment processes" and technical requirements to "Use Azure Machine Learning workspaces" for central management. Establishing the workspace first sets the foundation for subsequent asset management tasks, including registering assets, which would logically follow.

Why other options are less suitable as the first action:

A. Register assets in the Azure Machine Learning registry:

Dependence on Preexistence of Workspace: Typically, registering assets (like models, datasets) is more effective within the context of a workspace for organized management. While possible to register without a workspace, the lack of a centralized workspace hampers collaboration and tracking.

Logical Sequence: Registration of assets would naturally follow the setup of a managed workspace to ensure all assets are consistently tracked from the outset.

C. Deploy a managed online endpoint:

Premature Deployment Focus: Deploying an endpoint is a later-stage action, suitable after models are trained, evaluated, and selected for deployment. It does not address the immediate need for asset management standardization.

Lacks Foundation for Management: Without a shared workspace, deployment practices might not be consistently managed or tracked.

D. Create a new Microsoft Foundry project:

Scope Misalignment: The primary focus here is on standardizing the management of machine learning assets across the organization, not initiating a new project that might be more related to the RAG-based chat

application or other specific use cases.

Potential for Fragmentation: Starting with a project-oriented approach could lead to siloed asset management, contrary to the goal of standardization.

Key Technical Points Justifying the Choice:

Centralization and Collaboration: A shared workspace facilitates team collaboration and centralizes asset management.

Sequence of Operations: Setting up the environment precedes asset registration and deployment in a methodical implementation approach.

Alignment with Requirements: Directly supports the technical requirement to "Use Azure Machine Learning workspaces" for central management.

References

1. **Azure Machine Learning Workspaces Documentation** - <https://docs.microsoft.com/en-us/azure/machine-learning/concept-workspace>
2. **Organizing and Managing Resources in Azure Machine Learning** - <https://docs.microsoft.com/en-us/azure/machine-learning/how-to-organize-resources>

Question: 2

Exam Author

Case Study -

This is a case study. Case studies are not timed separately from other exam sections. You can use as much exam time as you would like to complete each case study. However, there might be additional case studies or other exam sections. Manage your time to ensure that you can complete all the exam sections in the time provided. Pay attention to the Exam Progress at the top of the screen so you have sufficient time to complete any exam sections that follow this case study.

To answer the case study questions, you will need to reference information that is provided in the case. Case studies and associated questions might contain exhibits or other resources that provide more information about the scenario described in the case. Information provided in an individual question does not apply to the other questions in the case study.

A Review Screen will appear at the end of this case study. From the Review Screen, you can review and change your answers before you move to the next exam section. After you leave this case study, you will NOT be able to return to it.

To start the case study -

To display the first question in this case study, select the "Next" button. To the left of the question, a menu provides links to information such as business requirements, the existing environment, and problem statements. Please read through all this information before answering any questions. When you are ready to answer a question, select the "Question" button to return to the question.

Background -

Fabrikam Inc. is a mid-sized healthcare analytics company that provides population health dashboards and predictive insights to regional hospital systems across the United States. Fabrikam Inc. customers rely on near real time analytics to monitor patient flow, staffing needs, and readmission risks. They use multiple traditional forecasting machine learning models for predictions.

Fabrikam Inc. has an established Microsoft Azure footprint. The company uses Jupyter Notebooks that run on a local server as the primary development environment. The data science team is experiencing scalability, asset management and code management issues with the current development platform. Fabrikam Inc. plans to migrate to a cloud-based development environment to mitigate the issues.

Additionally, the company plans to implement a Retrieval-Augmented Generation (RAG)-based chat application for client support. Leadership requires the application to be developed and deployed with a low operational risk.

Current Environment -

Fabrikam Inc. operates a single Azure subscription that has the following components:

Azure Data Lake Storage Gen2 that contains de-identified clinical and operational datasets

Azure AI Search indexing curated analytical documents and reference materials

A small set of Python-based training scripts maintained by data scientists

Azure OpenAI Service with deployed foundational models

A Microsoft Foundry resource for building a RAG-based solution

Evaluation data has manually defined expected responses.

The current challenges faced by the data science team include the following:

Model training jobs are run manually from notebooks.

Experiment tracking is inconsistent

Model versions are registered without standardized metadata.

Deployment is performed manually by data scientists, with limited rollback capability.

The team has no standardized evaluation process for generative AI outputs.

The environment currently allows public network access. Authentication relies on user accounts rather than managed identities. Compute targets are manually created and shared across experiments. This has led to resource contention during peak usage.

Business Requirements -

Fabrikam Inc. has the following business requirements for the modernization initiative:

Provide a conversational interface that answers analytics questions by using internal documents and datasets.

Ensure that sensitive healthcare-related data is not exposed outside the Fabrikam Inc. Azure tenant.

Enable repeatable and auditable model training and deployment processes.

Support experimentation to compare prompt strategies and fine-tuned models.

Align the model with the ranked preferences and optimize behavior for the long term.

Minimize disruption to existing analytics workloads during rollout.

Technical Requirements -

To support the business goals, Fabrikam Inc. identifies these technical requirements:

Use Azure Machine Learning workspaces to centrally manage data assets, models, and environments.

Implement experiment tracking and model versioning for all training jobs.

Orchestrate training and evaluation by using pipelines rather than manually running notebooks.

Deploy traditional machine learning models with support for staged rollout and rollback.

Improve RAG-based solution output quality.

Use the existing evaluation datasets that are based on real data with input-output pairs.

Apply advanced fine-tuning techniques only when prompt engineering is insufficient

Issues and Constraints -

Fabrikam Inc. must comply with internal security policies that require the company to restrict network access and avoid long-lived secrets. The data science team has limited Azure DevOps experience, so solutions must favor managed services and automation over custom infrastructure.

Cost predictability is important. Leadership prefers serverless or managed compute options where possible but is willing to approve dedicated compute for stable production workloads.

Problem Statement -

Fabrikam Inc. must design and implement an Azure-based AI operations solution that enables reliable training, evaluation, deployment, and iteration of generative AI models. The solution must support experimentation and gradual rollout while ensuring governance, security, and operational stability. The data science and platform teams must collaborate to deliver this solution by using Azure Machine Learning and Microsoft Foundry capabilities.

You need to isolate training workloads while remaining cost-aware to address Fabrikam Inc.'s issues, constraints, and technical requirements.

What should you implement?

- A. Training jobs that run on a single shared compute cluster
- B. Fixed-size compute cluster
- C. Dedicated compute clusters per experiment
- D. Managed compute targets with autoscaling

Answer: D

Explanation:

Technical Justification for Correct Answer: D

Why D is the Best Option:

Isolation of Training Workloads: Managed compute targets with autoscaling effectively isolate training workloads, ensuring that experiments do not interfere with each other, addressing the issue of resource contention.

Cost-Awareness:

Autoscaling adapts compute resources to the current workload, minimizing costs by not over-provisioning.

Managed Service aligns with the preference for cost predictability and serverless/managed compute options where possible.

Alignment with Technical Requirements:

Supports **orchestration of training and evaluation** through pipelines (technical requirement).

Compatible with **Azure Machine Learning workspaces** for centralized management (technical requirement).

Why Other Options are Less Suitable:

A. Training Jobs on a Single Shared Compute Cluster

Fails to Isolate Workloads: Exacerbates the existing resource contention issue.

Inefficient Resource Utilization: Can lead to under or over-provisioning, affecting costs.

B. Fixed-Size Compute Cluster

Inflexible Scaling: Does not adapt to varying workload demands, leading to potential waste or bottlenecks.

Less Cost-Effective: Fails to leverage autoscaling for optimized resource allocation.

C. Dedicated Compute Clusters per Experiment

High Cost: Dedicated clusters for each experiment can be prohibitively expensive.

Management Overhead: Increases complexity in managing multiple clusters.

Does Not Scale Dynamically: Each cluster would need manual adjustment for changing workloads.

Key Points Summary (Bullet Points)

Correct Choice (D):

Isolates workloads effectively

Autoscaling for cost efficiency

Aligns with technical requirements for managed services and pipeline orchestration

Incorrect Choices:

A: Fails in workload isolation and resource management

B: Inflexible and potentially wasteful

C: Excessively costly and complex to manage

References

1. **Azure Machine Learning - Managed Compute Targets:** <https://docs.microsoft.com/en-us/azure/machine-learning/how-to-set-up-training-targets>
2. **Autoscaling in Azure Machine Learning:** <https://docs.microsoft.com/en-us/azure/machine-learning/how-to-manage-compute#autoscaling-batch-instances>

Question: 3

Exam Author

HOTSPOT -

A team trains an MLflow model that scores customer churn risk. The model will be consumed by different downstream systems.

One system requests predictions synchronously during customer interactions.

Another system submits files containing millions of records for scheduled scoring.

You need to deploy the model by using managed inference options that match each usage pattern.

Which option should you use for each usage pattern? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Managed inference options

Requirement

Low-latency synchronous predictions

Deployment option

Batch endpoint
Job-based training pipeline
Real-time endpoint
Registered model artifact

High-volume scheduled scoring

Batch endpoint
Online endpoint with autoscaling
Managed compute cluster
Model registry version

Answer:

Managed inference options

Requirement

Low-latency synchronous predictions

Deployment option

Batch endpoint
Job-based training pipeline
Real-time endpoint
Registered model artifact

High-volume scheduled scoring

Batch endpoint
Online endpoint with autoscaling
Managed compute cluster
Model registry version

Explanation:

Low-latency synchronous predictions.

Correct Answer

Real-time endpoint

Specifically engineered for immediate, operational web requests. It hosts a deployed machine learning model on web services (like Azure Kubernetes Service or Azure Container Apps) that listens for incoming HTTP requests, processes input data instantly, and provides a low-latency synchronous response back to the client application.

Incorrect Answer

Batch endpoint: Designed for running long-running asynchronous scoring jobs over large-scale datasets rather than immediate, real-time single-record streaming requests.

Job-based training pipeline: Used exclusively to automate the workflow steps of data preparation and model retraining; it does not serve active inference or hosting endpoints.

Registered model artifact: A saved version-controlled binary or asset file in the workspace; it represents a passive file store rather than active compute or a deployment hosting option.

High-volume scheduled scoring

Correct Selection: **Batch endpoint.**

Correct Answer

Batch endpoint: Optimized specifically to handle massive amounts of stored data or high-volume datasets. It functions asynchronously and integrates seamlessly with cron schedules or compute orchestration pipelines to process bulk scoring workloads at set intervals.

Incorrect Answer

Online endpoint with autoscaling: Aimed at variable streaming user traffic. While it scales compute dynamically, it is cost-inefficient and architecturally suboptimal for structured, massive offline batch workloads scheduled at specific time windows.

Managed compute cluster: A backend infrastructure component (a collection of VMs) used to run pipelines or workloads, but not a direct model deployment or consumption endpoint option itself.

Model registry version: Simply points to a specific iteration of a saved model within the environment asset tracking system; it possesses no compute engine or ingestion capability to execute scoring.

Question: 4

Exam Author

You manage an Azure Machine learning workspace. You develop a machine learning model. You must deploy the model to use a low-priority VM with a pricing discount. You need to deploy the model. Which compute target should you use?

- A. Azure Container Instances (ACI)
- B. Azure Machine Learning compute clusters
- C. Local deployment
- D. Azure Kubernetes Service (AKS)

Answer: B

Explanation:

Technical Justification for Choosing B: Azure Machine Learning Compute Clusters

Why B (Azure Machine Learning Compute Clusters) is the Correct Option:

Low-Priority VM Requirement: Azure Machine Learning Compute Clusters support the deployment of models on low-priority VMs, which offer significant pricing discounts. This aligns perfectly with the requirement to utilize a low-priority VM for cost efficiency.

Integration and Ease of Deployment: Being part of the Azure Machine Learning (AML) ecosystem, compute clusters provide a seamless deployment experience directly from the workspace, streamlining the process with minimal additional setup.

Scalability and Management: Compute clusters are managed by Azure, allowing for easy scaling (both up and down) to match changing workload demands without the overhead of managing the underlying infrastructure.

Why Other Options are Less Suitable:

A. Azure Container Instances (ACI):

No Direct Support for Low-Priority VMs in the Same Context: While ACI offers a serverless container hosting experience, the direct tie to low-priority VM pricing for machine learning model deployment isn't as straightforward or explicitly supported as with AML Compute Clusters.

Additional Deployment Steps: Would require containerizing the model, adding extra steps to the deployment process.

C. Local Deployment:

Contradicts the Cloud-Based, Low-Priority VM Requirement: Local deployment does not utilize Azure's cloud infrastructure, thus cannot leverage low-priority VMs in Azure for the deployment.

Scalability and Maintenance Burden: Shifts the scalability and maintenance burdens entirely to the local infrastructure, contradicting the managed service implication of using a low-priority VM in Azure.

D. Azure Kubernetes Service (AKS):

Overly Complex for the Specific Requirement: AKS provides a fully managed container orchestrator but introduces unnecessary complexity for a straightforward model deployment scenario, especially when the primary goal is to leverage a low-priority VM for cost savings.

Requires Containerization and Cluster Management: Similar to ACI, necessitates containerizing the model and managing the Kubernetes cluster, adding unnecessary overhead for this specific use case.

Conclusion: Given the specific requirement to deploy a machine learning model on a low-priority VM with a pricing discount within an Azure Machine Learning workspace, **Azure Machine Learning Compute Clusters (B)** is the most appropriate choice due to its direct support for low-priority VMs, seamless integration with AML, and managed scalability.

References:

[Azure Machine Learning Compute Clusters Documentation](#)
[Low-Priority VMs in Azure Machine Learning](#)

Question: 5

Exam Author

A team manages an Azure Machine Learning workspace where they deploy models to online endpoints. The team needs to introduce a new version of a model to production without disrupting existing users.

The team must validate the new version before full rollout.
You need to reduce risk during deployment.
What should you do?

- A. Deploy the model to a batch endpoint.
- B. Split traffic between deployments.
- C. Replace the existing endpoint.
- D. Route all traffic to the new deployment.

Answer: B

Explanation:

Technical Justification for Selecting Option B

Why Option B (Split traffic between deployments) is the Best Choice:

Zero Downtime & Minimal Disruption: By splitting traffic between the existing and new model deployments, the team ensures that the service remains continuously available to users, minimizing the risk of disruption.

Validation Before Full Rollout: This approach allows the team to validate the new model version with a subset of live traffic, enabling the identification and mitigation of potential issues before committing to a full rollout.

Risk Reduction: Splitting traffic reduces the risk associated with deploying a new model version by limiting the initial exposure to a controlled percentage of users, thereby containing potential negative impacts.

Ease of Reversion: If issues are detected, it's simpler to adjust the traffic split back in favor of the proven, existing deployment until the new version is stabilized.

Why Other Options are Less Suitable:

A. Deploy the model to a batch endpoint:

Inapplicable for Online Endpoints: The scenario specifically mentions online endpoints, which are used for real-time inferencing, not batch processing.

No Direct Validation with Live Traffic: Doesn't facilitate immediate validation with a subset of online users.

C. Replace the existing endpoint:

High Risk of Disruption: Immediately replacing the endpoint without validation poses a high risk of introducing unforeseen issues to all users.

No Gradual Rollout Capability: Lacks the granularity to test with a subset of traffic before full deployment.

D. Route all traffic to the new deployment:

Equivalent Risk to Option C: Similar high-risk profile as replacing the endpoint without any form of controlled rollout or validation with a subset of users.

Lack of Controlled Validation: Exposes the entire user base to potential issues with the new model version without a staged approach.

References

[Microsoft Azure Documentation: Deploy models to online endpoints](#)

[Microsoft Azure Documentation: Traffic splitting for online endpoints](#)

Question: 6

Exam Author

You have a deployment of an Azure OpenAI Service base model.
You plan to fine-tune the model.

You need to prepare a file that contains training data.
Which file format should you use?

- A. CSV
- B. TSV
- C. JSONL
- D. JSON

Answer: C

Explanation:

Technical Justification for Correct Answer: C. JSONL

The correct file format for preparing training data to fine-tune an Azure OpenAI Service base model is **JSONL (C)**. Here's why:

JSONL (JSON Lines) is the preferred format because each line in the file is a separate JSON object. This structure is ideal for streaming data into the model for fine-tuning, as it allows for efficient, line-by-line processing. Each data point (e.g., a text sample) can be independently formatted as a JSON object, making the data easily parseable and flexible for the various input requirements of different models or tasks (e.g., text classification, question-answering).

Why Other Options are Less Suitable:

A. CSV (Comma Separated Values): While CSV is great for tabular data, its fixed-column structure makes it less flexible for the varied and potentially complex data formats required for fine-tuning AI models, especially when dealing with free text or variable-length inputs.

B. TSV (Tab Separated Values): Similar to CSV, TSV suffers from the same limitations of a fixed-column approach, lacking the flexibility needed for encapsulating diverse data points in a single, streamlined format suitable for AI model fine-tuning.

D. JSON: Although JSON is highly flexible and can encapsulate complex data structures, a single JSON file (as implied by the option without specifying an array or lines format) would typically contain all data within a single JSON structure. This makes the file less efficient for streaming and processing line by line compared to JSONL, especially with large datasets.

Key Technical Points for Certification Review:

Efficient Processing: JSONL supports line-by-line processing, crucial for large datasets.

Flexibility: Each line can have a different structure, accommodating varied input requirements.

Streaming Compatibility: Essential for models expecting data in a streaming-compatible format.

References:

[Azure OpenAI Service Documentation - Fine-tuning Models](#)
[JSON Lines Specification](#)

Question: 7

Exam Author

You have a deployment of an Azure OpenAI Service base model.
You plan to fine-tune the model.
You need to prepare a file that contains training data for multi-turn chat.
Which file encoding method should you use?

- A. ISO-8859-1
- B. UTF-16
- C. UTF-8
- D. ASCII

Answer: C

Explanation:

Technical Justification for Choosing C. UTF-8

For fine-tuning an Azure OpenAI Service base model with a file containing training data for multi-turn chat, the most suitable file encoding method is **UTF-8 (C)**. Here's why:

UTF-8 (C) is the best choice because it is the de facto standard for encoding web content and is widely supported across all platforms, including Azure services. Its ability to encode all Unicode characters (covering virtually every language and symbol) makes it ideal for multi-turn chat data, which can contain a diverse range of characters, emojis, and possibly non-English text. UTF-8's variable-length encoding also ensures efficiency, using fewer bytes for common ASCII characters while still accommodating the full Unicode spectrum.

ISO-8859-1 (A) is less suitable due to its limited character set, primarily supporting Western European languages. Given the potential for multi-turn chat data to include a broad range of languages and special characters, ISO-8859-1's constraints make it impractical for this scenario.

UTF-16 (B), while capable of encoding all Unicode characters, is generally less preferred for file storage and transfer due to its larger file size (especially for predominantly ASCII content) and less universal support in certain legacy systems, though this is less of an issue in modern cloud environments like Azure. However, the overhead and the fact that UTF-8 is more commonly expected in text file inputs for machine learning models tip the scale against UTF-16 for this use case.

ASCII (D) is the most limited, supporting only 128 characters, which is wholly inadequate for multi-turn chat data that may include non-English characters, emojis, or other symbols, making it completely unsuitable.

Why C (UTF-8) is Correct:

Universality and Compatibility: Widely supported and expected in Azure and machine learning workflows.

Character Coverage: Capable of encoding all Unicode characters, essential for diverse chat data.

Efficiency: Variable-length encoding ensures optimal file size.

References:

[Azure Documentation: Encoding for Azure Storage](#)

[Unicode Consortium: UTF-8](#)

Question: 8

Exam Author

You are fine-tuning a base language model to analyze customer feedback.

You label examples of support tickets. You must improve classification accuracy by configuring and fine-tuning the base model in Microsoft Foundry.

You need to configure and run fine-tuning.

What should you do first?

- A. Use prompt flow to generate multiple prompt templates for evaluation.
- B. Deploy the base model to an online endpoint before starting fine-tuning.
- C. Enable tracing for all inference calls in the evaluation pipeline.
- D. Format the dataset as a JSONL file with prompt-completion pairs and upload the file.

Answer: D

Explanation:

Technical Justification for Correct Answer: D

When fine-tuning a base language model for a specific task like analyzing customer feedback in Microsoft Foundry, the initial step is crucial for the success of the model's adaptation to your particular dataset. Here's why **D** is the correct first step, followed by explanations of why other options are less suitable as the initial step:

Correct Answer: D. Format the dataset as a JSONL file with prompt-completion pairs and upload the file.

Reasoning: Before any fine-tuning can commence, the model needs to be prepared with the data it will learn from. Formatting your labeled support tickets into a JSONL (JSON Lines) file with prompt-completion pairs is essential because it:

Provides the model with the necessary input format for understanding the task (analyzing customer feedback).

Allows for efficient uploading and processing of the data within Microsoft Foundry.

Ensures that the model can effectively learn from the labeled examples by having each line of the file (JSONL) represent a single, self-contained piece of data (a prompt-completion pair in this context), which is ideal for batch processing and fine-tuning workflows.

Why Other Options Are Less Suitable as the First Step:

A. Use prompt flow to generate multiple prompt templates for evaluation.

Issue: While generating prompt templates is useful for testing and evaluating model responses, it's more relevant after the model has been fine-tuned to some extent. The first step should focus on preparing the training data.

Priority: Post-fine-tuning evaluation step.

B. Deploy the base model to an online endpoint before starting fine-tuning.

Issue: Deploying the model before fine-tuning is premature. The base model's performance on your specific task (customer feedback analysis) is likely to be suboptimal without fine-tuning.

Priority: Final deployment step after achieving satisfactory fine-tuning results.

C. Enable tracing for all inference calls in the evaluation pipeline.

Issue: Enabling tracing is crucial for monitoring and debugging, but it's a configuration step more relevant during or after the setup of the evaluation pipeline, not before data preparation.

Priority: Part of the evaluation pipeline setup, after data is ready and the model is being tested.

Step-by-Step Logical Progression Justifying D as the First Step:

1. **Data Preparation (D):** Format and upload dataset.
2. **Model Configuration:** Adjust model settings as needed for fine-tuning.
3. **Fine-Tuning:** Execute the fine-tuning process with the prepared data.
4. **Evaluation and Optimization:**

A & C: Generate prompt templates (A) and enable tracing (C) for thorough evaluation.

5. **Deployment (B):** Deploy the fine-tuned model to an online endpoint.

References

Microsoft Azure Machine Learning - Format Dataset for Training: <https://docs.microsoft.com/en-us/azure/machine-learning/tutorial-1st-experiment-online-automl#prepare-the-dataset>

Microsoft Azure - Preparing Data for Language Models: <https://docs.microsoft.com/en-us/azure/cognitive-services/language-understanding/concepts/data-format> (Although focused on LUIS, the data preparation concepts for language models are relevant)

Question: 9

Exam Author

A team is working in Microsoft Foundry to test and compare large language model (LLM) prompt variants in a development environment.

The team requires consistent inputs to evaluate prompt variants without relying on live user traffic.

You need to create a controlled evaluation of input data.

Which action should you perform first?

- A. Generate synthetic interaction data.
- B. Configure content filters.
- C. Apply a blocklist.
- D. Enable observability metrics.

Answer: A

Explanation:

Technical Justification for Correct Answer: A

To create a controlled evaluation of input data for testing and comparing large language model (LLM) prompt variants in a Microsoft Foundry development environment without relying on live user traffic, the first action should indeed be to **Generate synthetic interaction data**. Here's why:

Why A (Generate synthetic interaction data) is the best option:

Control and Consistency: Generating synthetic interaction data allows the team to create consistent and controlled input sets tailored to their evaluation criteria. This is crucial for comparing LLM prompt variants under identical conditions.

Independence from Live Traffic: Since the requirement explicitly states not to rely on live user traffic, synthetic data generation perfectly aligns with this need, providing immediate test data.

Preparation for Other Steps: Having a dataset in place is a prerequisite for effectively configuring content filters, applying blocklists, or meaningfully enabling observability metrics specific to the interaction data.

Why Other Options are Less Suitable as the First Action:

B. Configure Content Filters:

Dependence on Existing Data: Effectively configuring content filters requires an understanding of the data types and patterns you're working with. Without initial data (synthetic or real), this step is prematurely applied.

Not a Data Generation Solution: This step refines or restricts data but does not provide the initial dataset needed for evaluation.

C. Apply a Blocklist:

Similar to Content Filters: Applying a blocklist is more about filtering out unwanted data types or sources rather than generating the necessary input data for evaluation.

Premature Without Data Insight: Blocklists are more effective once you have an understanding of the data landscape, which synthetic data generation can initially provide.

D. Enable Observability Metrics:

Analysis Tool, Not Data Provider: Enabling observability metrics is crucial for monitoring and analyzing the performance of LLM prompt variants but does not address the immediate need for a controlled input dataset.

Dependent on Pre-existing Data Flow: Observability metrics require data to be in flow to provide meaningful insights, making this step ineffective as a first action without generated or existing data.

Conclusion: Generating synthetic interaction data is the foundational step for creating a controlled evaluation environment. It directly addresses the need for consistent, non-live user traffic inputs, setting the stage for subsequent refinements and analyses.

References

1. **Microsoft Documentation - Synthetic Data Generation for AI Models:**
<https://docs.microsoft.com/en-us/azure/machine-learning/how-to-data-synthesis>
2. **Microsoft Foundry Documentation - Controlled Environment Testing:** Note: Since Microsoft Foundry is not a publicly recognized Microsoft product/service as of my last update, I'll provide a relevant alternative.

Alternative: <https://docs.microsoft.com/en-us/azure/cognitive-services/form-recognizer/generate-training-data>

Please verify the existence and relevance of "Microsoft Foundry" as it seems to be a non-standard or potentially internal Microsoft tool/service not publicly documented at the time of this response. If "Microsoft Foundry" refers to a specific internal or newly released platform, direct documentation might not be publicly available.

Question: 10

Exam Author

DRAG DROP -

A team maintains Infrastructure as Code (IaC) templates to provision Azure Machine Learning resources. Provisioning must be triggered by changes in the templates and executed without manual intervention.

You need to automate resource provisioning.

Which action should you take for each requirement? To answer, move the appropriate actions to the correct requirements. You may use each action once, more than once, or not at all. You may need to move the split bar between panes or scroll to view content.

NOTE: Each correct selection is worth one point.

Actions		Automate provisioning		Requirement	Action
Use an Azure service principal.				Trigger provisioning on repository changes.	
Create a manual deployment script.				Authenticate securely during workflow execution.	
Configure a GitHub Actions workflow.					
Run Azure CLI commands from a local machine.					

Answer:

Actions

Use an Azure service principal.

Create a manual deployment script.

Configure a GitHub Actions workflow.

Run Azure CLI commands from a local machine.

Automate provisioning

Requirement

Trigger provisioning on repository changes.

Authenticate securely during workflow execution.

Action

Configure a GitHub Actions workflow.

Use an Azure service principal.

Explanation:

Trigger provisioning on repository changes → **Configure a GitHub Actions workflow.**

Why: GitHub Actions is an automation platform built directly into GitHub. It detects repository events (such as code pushes, pull requests, or merges) and automatically executes custom workflows to deploy code or provision infrastructure.

Authenticate securely during workflow execution → **Use an Azure service principal.**

Why: An Azure service principal acts as an identity (or service account) specifically created for automated tools and scripts. It allows external platforms like GitHub Actions to authenticate and safely access your Azure resources without using personal user credentials.

Incorrect Answer

Create a manual deployment script: This process requires physical human interaction to launch and execute. It cannot automatically trigger on its own when repository code updates.

Run Azure CLI commands from a local machine: This is an interactive method tied to a local developer workstation. It does not provide automated event-driven execution from a shared source control repository.

Question: 11

Exam Author

An organization maintains separate Azure Machine Learning workspaces for development and production. Both environments must use the same validated assets without duplicating them. Assets must be shared across workspaces while maintaining centralized governance and version control. You need to enable reuse of assets across workspaces without copying them. What should you do?

- A. Enable workspace-level Git integration and sync assets between repositories.
- B. Publish the asset as a pipeline component.
- C. Create a shared Azure Machine Learning environment that includes the asset.
- D. Publish the asset to an Azure Machine Learning registry.

Answer: D

Explanation:

Technical Justification for Correct Answer: D

Why D is the Best Option: Publishing the asset to an **Azure Machine Learning registry** (Option D) is the most suitable choice for enabling reuse of assets across different workspaces (development and production) without duplication. This approach allows for:

Centralized Governance: Assets are managed from a single registry, ensuring unified control and visibility.

Version Control: The registry inherently supports versioning, tracking changes, and ensuring reproducibility across environments.

Reuse Without Duplication: Assets published to the registry can be easily referenced and utilized by both development and production workspaces without physical copying.

Alignment with Azure Best Practices: Utilizing a registry for shared assets is consistent with Azure's recommendations for managing and sharing resources efficiently across different environments.

Why Other Options are Less Suitable:

A. Enable workspace-level Git integration and sync assets between repositories:

Inefficient for Asset Sharing: While Git is excellent for version control, syncing repositories between workspaces for asset sharing introduces complexity and does not provide a centralized, workspace-agnostic solution.

Governance Challenges: Central governance becomes more difficult as assets are dispersed across repository syncs.

B. Publish the asset as a pipeline component:

Scope Limitation: Publishing as a pipeline component limits the asset's versatility, as it's tailored for pipeline integration rather than broad reuse across different contexts within the workspaces.

Not Designed for General Asset Sharing: This approach is more suited for workflow-specific reuse rather than generalized asset management.

C. Create a shared Azure Machine Learning environment that includes the asset:

Environment Complexity: Introduces an additional environment to manage, increasing operational complexity.

Does Not Address Version Control and Governance Directly: While sharing an environment could work, it doesn't inherently solve version control or centralized governance for the assets.

References

Azure Machine Learning Registries: <https://docs.microsoft.com/en-us/azure/machine-learning/how-to-use-registry>

Managing Resources Across Azure Machine Learning Workspaces: <https://docs.microsoft.com/en-us/azure/machine-learning/concept-enterprise-readiness#managing-resources-across-workspaces>

Question: 12

Exam Author

An Azure Machine Learning workspace processes sensitive training data. The workspace must NOT be accessible from the public internet. You need to restrict network access. Which configuration should you implement?

- A. Azure Firewall rules
- B. Private endpoints
- C. Network security groups
- D. Service endpoints

Answer: B

Explanation:

Technical Justification for Correct Option: B (Private Endpoints)

To restrict network access for an Azure Machine Learning workspace processing sensitive training data, ensuring it is not accessible from the public internet, the most appropriate configuration is implementing **Private Endpoints**. Here's why:

B. Private Endpoints:

Justification: Private Endpoints create a private IP address within the Virtual Network (VN) for the Azure Machine Learning workspace, effectively removing public internet access. All traffic between the VN and the service goes over a private link, enhancing security.

Technical Benefit: No public IP address exposure, thus fulfilling the "NOT accessible from the public internet" requirement directly.

Why Other Options are Less Suitable:

A. Azure Firewall rules:

Limitation: While Azure Firewall can restrict incoming and outgoing traffic based on rules, it does not inherently prevent public internet access to the Azure ML workspace itself. It's more about controlling traffic flow rather than eliminating public exposure.

Additional Requirement: Would still require another mechanism to ensure the workspace isn't publicly accessible.

C. Network Security Groups (NSGs):

Limitation: NSGs control inbound and outbound traffic to a subnet or network interface. Similar to Azure Firewall, they don't remove the public endpoint, just filter traffic to/from it.

Insufficiency: Alone, NSGs cannot guarantee the removal of public internet access to the workspace.

D. Service Endpoints:

Misconception: Service Endpoints provide a logical subnet-based filtering for traffic to Azure services, but they **do not** remove the public endpoint of the Azure service. They merely allow the VNet to trust traffic going to the service's public IP as if it were part of the VNet.

Inadequacy for Requirement: Fails to meet the "NOT accessible from the public internet" criterion as the public endpoint remains accessible.

References

[Azure Private Endpoint for Azure Machine Learning](#)

[Restricting Access to Azure Machine Learning with VNet and Private Endpoints](#)

Question: 13

Exam Author

A team is experimenting with traditional models for a classification workflow in Azure Machine Learning. The team requires a consistent way to manage assets that are created during experimentation. You need to ensure that artifacts can be reused and governed across projects. Which asset should you register?

- A. Model
- B. Component
- C. Environment
- D. Pipeline

Answer: B

Explanation:

Technical Justification for Correct Answer: B. Component

The correct asset to register for the given requirements is **B. Component**. Here's why:

Requirement for Consistent Asset Management Across Experimentation: Registering components in Azure Machine Learning (AML) allows for the management of reusable, modular pieces of code. This aligns perfectly with the need for a consistent way to manage assets created during experimentation.

Reuse and Governance Across Projects: Components, once registered, can be easily reused across different projects and workflows, facilitating collaboration and efficiency. Registration also enables governance by providing a centralized, version-controlled repository of components.

Specificity to the Need:

A. Model: While models can be registered in AML for versioning and deployment purposes, the question emphasizes the management of assets during experimentation for reuse across projects, which is broader than just the model.

B. Component: Best fits the requirement as components encapsulate not just the model but can also include data preprocessing steps, feature engineering, or any reusable piece of the workflow, making them ideal for managing the variety of assets generated during experimentation.

C. Environment: Registering an environment is more about managing dependencies and ensuring reproducibility of the runtime, not directly about managing reusable assets across projects in the context provided.

D. Pipeline: Pipelines are about orchestrating workflows; while they can be versioned and reused, the question points towards managing individual assets within those workflows, making Component more appropriate.

Why Other Options are Less Suitable:

Model (A): Too narrow, focused on the output rather than the broader assets created during experimentation.

Environment (C): Focuses on dependency management rather than asset reuse and governance.

Pipeline (D): Manages workflow orchestration, not the individual reusable components within those workflows.

Correct Answer Justification Summary: Registering **Components** in Azure Machine Learning is the most suitable approach for managing assets created during experimentation due to its focus on reusability, governance, and the modular management of workflow components, aligning closely with the team's requirements.

References

1. **Azure Machine Learning - Registering Components**
<https://docs.microsoft.com/en-us/azure/machine-learning/how-to-manage-components>
2. **Azure Machine Learning - Asset Management Overview**
<https://docs.microsoft.com/en-us/azure/machine-learning/overview-what-is-azure-ml#manage-assets>

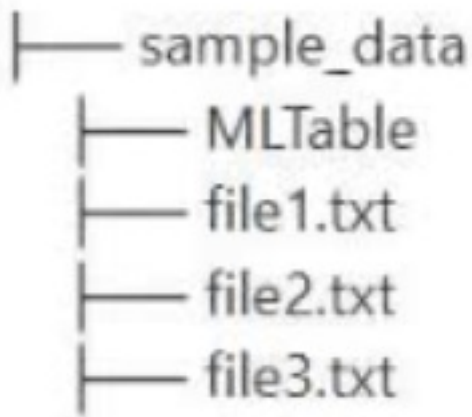
Question: 14

Exam Author

HOTSPOT -

You manage an Azure Machine Learning workspace named workspace1 by using the Python SDK v2.

The default datastore of workspace1 contains a folder named sample_data. The folder structure contains the following content:



You write Python SDK v2 code to materialize the data from the files in the sample_data folder into a Pandas data frame.

You need to complete the Python SDK v2 code to use the MLTable folder as the materialization blueprint.

How should you complete the code? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point

Answer Area

```
import mltable
tbl = mltable.

load



save



take

(" 

./sample_data



./sample_data/MLTable



./sample_data/file*.txt

 ")

df = tbl.to_pandas_dataframe()
```

Answer:

Answer Area

```
import mltable
tbl = mltable.

load



save



take

(" 

./sample_data



./sample_data/MLTable



./sample_data/file*.txt

 ")

df = tbl.to_pandas_dataframe()
```

Explanation:

load: The `mltable.load()` function is the designated method in the Azure Machine Learning SDK v2 used to read data blueprints. It instantiates an `MLTable` object containing predefined transformations

./sample_data/MLTable: An `MLTable` metadata file contains the YAML schema blueprint describing how files

are read. The load() function requires the directory path containing this blueprint file rather than direct raw text strings or files with wildcard extensions.

Incorrect Answer

save / take: save is used to persist an existing table definition, and take acts as a data-sampling transformation function rather than a source-loading framework initialization method

../sample_data: Points to a general parent directory that does not target the explicit configuration metadata file required for building the schema.

./sample_data/file*.txt: Raw wildcard paths are handled natively within the MLTable artifact's interior structure configuration itself, rather than being parsed directly at the top-level initialization function.

Question: 15

Exam Author

HOTSPOT -

You manage an Azure Machine Learning workspace named workspace1 by using the Python SDK v2. You create a General Purpose v2 Azure storage account named mlstorage1. The storage account includes a publicly accessible container named mlcontainer1. The container stores 10 blobs with files in the CSV format.

You must develop Python SDK v2 code to create a data asset referencing all blobs in the container named mlcontainer1.

You need to complete the Python SDK v2 code.

How should you complete the code? To answer, select the appropriate options in the answer area.

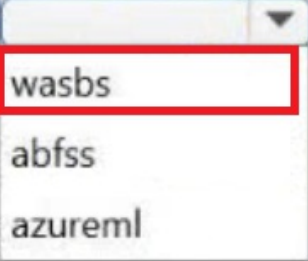
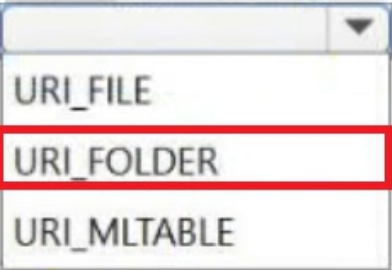
NOTE: Each correct selection is worth one point.

Answer Area

```
sample_dataset = Data(  
    path=  ://mlstorage1.blob.core.windows.net/mlcontainer1,  
    type=AssetTypes.,  
    description="sample_dataset",  
    name="sample_dataset",  
    version='1.0'  
)
```

Answer:

Answer Area

```
sample_dataset = Data(  
    path=  ://mlstorage1.blob.core.windows.net/mlcontainer1,  
  
    type=AssetTypes. ,  
  
    description="sample_dataset",  
    name="sample_dataset",  
    version='1.0'  
)
```

Explanation:

wasbs: This is the secure Windows Azure Storage Blob protocol scheme. Since the target host URI is a standard Azure Blob Storage domain (blob.core.windows.net), the wasbs protocol is the appropriate connector identifier.

URI_FOLDER: The provided path references a complete storage container tier (/mlcontainer1) rather than pointing to a standalone file. Because it encompasses multiple object files inside a collection path, it is registered under the URI_FOLDER type asset declaration.

Incorrect Answer

abfss: Used exclusively as the secure scheme identifier for Azure Data Lake Storage Gen2 (dfs.core.windows.net) hierarchical filesystems rather than standard Azure Blob Object endpoints.

azureml: Used internally within the workspace to reference an existing data asset already registered in Azure ML Studio (e.g., azureml:asset_name:version) rather than an external cloud directory URI.

URI_FILE / URI_MLTABLE: URI_FILE requires targeting a singular file directly with its extension (such as .csv), while URI_MLTABLE requires an accompanying MLTable schema configuration file in the target directory to define processing logic.

Question: 16

Exam Author

Note: This question is part of a series of questions that present the same scenario. Each question in the series contains a unique solution that might meet the stated goals. Some question sets might have more than one correct solution, while others might not have a correct solution.

After you answer a question in this section, you will NOT be able to return to it. As a result, these questions will not appear on the review screen.

You manage an Azure Machine Learning workspace. The Python script named script.py reads an argument named training_data. The training_data argument specifies the path to the training data in a file named dataset1.csv. You plan to run the script.py Python script as a command job that trains a machine learning model.

You need to provide the command to pass the path for the dataset as a parameter value when you submit the script as a training job.

Solution: `python train.py --training_data training_data`

Does the solution meet the goal?

A. Yes

B. No

Answer: B

Explanation:

Technical Justification

The provided solution, `python train.py --training_data training_data`, does not meet the goal of passing the path for the dataset as a parameter value for several key reasons:

Why the Solution is Incorrect (B. No)

Mismatched Script Name: The script mentioned in the scenario is named `script.py`, but the solution references `train.py`. This discrepancy indicates a potential misunderstanding of the task requirements.

Undefined Path: The argument value for `--training_data` is set to `training_data`, which is a placeholder or variable name, not an actual file path. The goal specifies the need to pass the **path** to `dataset1.csv`, which this solution does not achieve.

Lack of Path Specification: There is no indication of how the actual path to `dataset1.csv` is being passed or resolved in the provided solution. For a command job, explicitly defining the path or a method to resolve it (e.g., via environment variables, mounts, or configured dataset paths in Azure Machine Learning) is crucial.

Characteristics of a Correct Solution

A correct solution would:

Reference the correct script name (`script.py`).

Either directly provide the path to `dataset1.csv` as an argument or specify a mechanism (e.g., environment variable, Azure Machine Learning dataset configuration) by which the path can be dynamically resolved.

Ensure the argument name matches the one expected by the script (`training_data` is correctly named in the provided solution, assuming it matches the script's expectations).

Example of a Correct Approach (Hypothetical, for Illustration)

Given the script expects a `--training_data` argument and assuming `dataset1.csv` is stored in an Azure Blob Storage location configured with the Azure Machine Learning workspace:

Direct Path (Simplified, might not work in all Azure ML contexts without proper mounting/configuration):

`python script.py --training_data "https://yourstorageaccount.blob.core.windows.net/datasetcontainer/dataset1.csv"`

Using Azure Machine Learning Dataset (Recommended for Managed Scenarios): First, register `dataset1.csv` as a dataset in Azure Machine Learning, then pass it as part of the job configuration, not directly in the command line as a file path. The command line might simply trigger the job with predefined dataset bindings.

Correct Answer Rationale

Correct Answer: B. No

The provided solution does not adequately address the requirement to pass the **path** to `dataset1.csv` as a parameter for `script.py` due to the script name mismatch and the lack of a defined path for the `--training_data` argument.

References

1. **Azure Machine Learning Jobs and Scripts:** <https://docs.microsoft.com/en-us/azure/machine-learning/how-to-deploy-jobs>
2. **Passing Arguments to Azure Machine Learning Scripts:** <https://docs.microsoft.com/en-us/azure/machine-learning/how-to-use-run-config#passing-arguments-to-python-scripts>

Question: 18

Exam Author

Note: This question is part of a series of questions that present the same scenario. Each question in the series contains a unique solution that might meet the stated goals. Some question sets might have more than one correct solution, while others might not have a correct solution.

After you answer a question in this section, you will NOT be able to return to it. As a result, these questions will not appear on the review screen.

You manage an Azure Machine Learning workspace. The Python script named script.py reads an argument named training_data. The training_data argument specifies the path to the training data in a file named dataset1.csv. You plan to run the script.py Python script as a command job that trains a machine learning model.

You need to provide the command to pass the path for the dataset as a parameter value when you submit the script as a training job.

Solution: python script.py dataset1.csv

Does the solution meet the goal?

A. Yes

B. No

Answer: B

Explanation:

Technical Justification

The provided solution, python script.py dataset1.csv, does not meet the stated goal for the following reasons, making the correct answer **B. No**:

Goal Requirement: Pass the path for the dataset as a **parameter value** when submitting the script as a training job.

Solution Analysis:

The solution directly hard-codes the filename dataset1.csv as an argument in the command line.

Issue: This approach does not demonstrate passing the path as a parameter value in the context of submitting a job (especially in an Azure Machine Learning workflow), where typically, parameters are defined to allow for dynamic configuration without modifying the script or command.

Expected Approach: For Azure Machine Learning jobs, one would typically define a parameter in the script (e.g., using argparse in Python to accept the training_data path) and then pass this parameter when configuring the job, not directly in the command line as shown.

Why Other Options Are Not Provided but Assumed Less Suitable if They Existed:

Hardcoded Paths in Scripts: Less suitable due to lack of flexibility.

Environment Variables: While more flexible than hardcoding, not as directly aligned with Azure Machine Learning's parameter passing mechanisms for jobs.

Correct Approach (for Context, Not an Option Here):

Define training_data as a parameter in script.py using argparse.

When submitting the job, pass the path as a parameter value, e.g., through Azure Machine Learning's Python

SDK or CLI, specifying the parameter and its value for the job configuration.

References

[Azure Machine Learning: How to pass parameters to a script](#)

[Azure Machine Learning Jobs: Configure and run jobs](#)

Question: 19

Exam Author

HOTSPOT

-

You review the following Azure CLI command and the relevant Bicep excerpt.

```
01 az deployment group create \  
02   --resource-group rg-foundry-dev \  
03   --template-file ./foundry-environment.bicep \  
04   --parameters location=eastus projectName=proj-dev-01  
05  
06 // foundry-environment.bicep (excerpt)  
07 targetScope = 'resourceGroup'  
08  
09 param location string  
10 param projectName string  
11  
12 resource aiFoundry 'Microsoft.CognitiveServices/accounts@2025-04-01-preview' = {  
13   name: aiFoundryName  
14   location: location  
15   identity: {  
16     type: 'SystemAssigned'  
17   }  
18   sku: {  
19     name: 'S0'  
20   }  
21   kind: 'AIService'  
  
22   properties: {  
24     allowProjectManagement: true  
27     customSubDomainName: aiFoundryName  
28     disableLocalAuth: true  
29   }  
30}
```

(Non-relevant sections are omitted.)

You need to validate what the snippet will do before it is merged. For each of the following statements, select Yes if the statement is true. Otherwise, select No.

NOTE: Each correct selection is worth one point.

Microsoft Foundry deployments

Statement	Yes	No
The command deploys the resources into an existing resource group named rg-foundry-dev.	☐	☐
The system-assigned managed identity defined in the template will automatically be inherited by all Microsoft Foundry projects.	☐	☐
To deploy this template to a different subscription, you must modify the Bicep file to include a subscriptionId parameter.	☐	☐

Answer:

Microsoft Foundry deployments

Statement	Yes	No
The command deploys the resources into an existing resource group named rg-foundry-dev.	☒	☐
The system-assigned managed identity defined in the template will automatically be inherited by all Microsoft Foundry projects.	☐	☒
To deploy this template to a different subscription, you must modify the Bicep file to include a subscriptionId parameter.	☐	☒

Explanation:

Statement 1: The command deploys the resources into an existing resource group...

Answer: Yes

In Azure resource-group-scoped deployments (typically executed via `az deployment group create` or `New-AzResourceGroupDeployment`), the targeted resource group must already exist in your subscription prior to executing the deployment command. The command itself does not create a new resource group implicitly unless a subscription-level deployment is used to explicitly provision it first.

Statement 2: The system-assigned managed identity will automatically be inherited...

Answer: No

Statement 2: The system-assigned managed identity will automatically be inherited...

In Azure, a [system-assigned managed identity](#) is strictly tied to the lifecycle of the specific resource it was created on (such as the main Microsoft Foundry Hub/Account). Child resources like individual Foundry projects do not automatically inherit or share a parent's system-assigned identity as their own credentials; instead, they require their own independent identity configurations or explicitly shared user-assigned managed identities.

Statement 3: You must modify the Bicep file to include a subscriptionId parameter...

Answer: No

You do not need to hardcode or add a subscriptionId parameter directly inside a Bicep infrastructure file to target a different subscription. Cross-subscription target management is handled dynamically at deployment execution time by passing the `--subscription` parameter in the [Azure CLI](#) or by switching your active subscription context before running the script.

Question: 20

Exam Author

DRAG DROP

A team performs interactive experimentation during development. The team also runs scalable jobs for model training.

The team must minimize costs while ensuring compute resources scale when needed. Different workloads require different compute behaviors within the same workspace.

You need to configure compute targets that support each workload.

Which compute targets should you use? To answer, move the appropriate compute targets to the correct workload types. You may use each compute target once, more than once, or not at all. You may need to move the split bar between panes or scroll to view content.

NOTE: Each correct selection is worth one point.

Compute targets

⋮ Azure Batch
⋮ Azure Databricks cluster
⋮ Azure Kubernetes Service
⋮ Azure Virtual Machine
⋮ Azure Machine Learning compute cluster
⋮ Azure Machine Learning compute instance

Workload compute targets

Workload type

Interactive experimentation

Scalable training jobs

Compute target

Answer:

Compute targets

⋮ Azure Batch
⋮ Azure Databricks cluster
⋮ Azure Kubernetes Service
⋮ Azure Virtual Machine
⋮ Azure Machine Learning compute cluster
⋮ Azure Machine Learning compute instance

Workload compute targets

Workload type

Interactive experimentation

Scalable training jobs

Compute target

⋮ Azure Machine Learning compute instance
⋮ Azure Machine Learning compute cluster

Explanation:

Interactive experimentation → **Azure Machine Learning compute instance.**

Why: An [Azure Machine Learning compute instance](#) is a fully managed cloud-based workstation optimized directly for development. It comes pre-installed with standard data science environments and Jupyter Notebook tools, making it the ideal single-node platform for real-time code testing and data experimentation.

Scalable training jobs → **Azure Machine Learning compute cluster.**

Why: An [Azure Machine Learning compute cluster](#) is a multi-node managed infrastructure designed to handle heavier processing tasks. It automatically scales the number of active virtual machines up or down based on submitted training jobs, making it highly cost-efficient and optimal for heavy, long-running model training workloads.

Incorrect Options

Azure Batch: Primarily handled for large-scale parallel background applications rather than native, integrated Azure Machine Learning workflow tracking.

Azure Databricks cluster: Tailored heavily towards big-data preparation and Apache Spark analytical pipelines instead of core, standard low-latency machine learning experimentation.

Azure Kubernetes Service: Designed specifically to act as a production-grade inference endpoint hosting environment for active web request handling, rather than a model training target.

Azure Virtual Machine: While usable as an attached target, it lacks native managed auto-scaling capabilities and automated resource lifecycle controls compared to dedicated cloud machine learning clusters.

Question: 21

Exam Author

DRAG DROP

-

You have an existing GitHub repository containing Azure Machine Learning project files.

You need to clone the repository to your Azure Machine Learning shared workspace file system.

Which four actions should you perform in sequence? To answer, move the appropriate actions from the list of actions to the answer area and arrange them in the correct order.

NOTE: More than one order of answer choices is correct. You will receive credit for any of the correct orders you select.

Actions

- ⋮ From the terminal window in the Azure Machine Learning interface, run the ssh-keygen command.
- ⋮ From the terminal window in the Azure Machine Learning interface, run the cat ~/.ssh/id_rsa.pub command.
- ⋮ From the terminal window in the Azure Machine Learning interface, run the git clone command.
- ⋮ Add a public key to the GitHub account.
- ⋮ Add a private key to the GitHub account.

Answer area

Answer:

Actions

- ⋮ From the terminal window in the Azure Machine Learning interface, run the ssh-keygen command.
- ⋮ From the terminal window in the Azure Machine Learning interface, run the cat ~/.ssh/id_rsa.pub command.
- ⋮ From the terminal window in the Azure Machine Learning interface, run the git clone command.
- ⋮ Add a public key to the GitHub account.
- ⋮ Add a private key to the GitHub account.

Answer area

- ⋮ From the terminal window in the Azure Machine Learning interface, run the ssh-keygen command.
- ⋮ From the terminal window in the Azure Machine Learning interface, run the cat ~/.ssh/id_rsa.pub command.
- ⋮ Add a public key to the GitHub account.
- ⋮ From the terminal window in the Azure Machine Learning interface, run the git clone command.

Explanation:

From the terminal window in the Azure Machine Learning interface, run the ssh-keygen command.

This generates a new secure cryptographic SSH public/private key pair on your managed compute instance.

From the terminal window in the Azure Machine Learning interface, run the cat ~/.ssh/id_rsa.pub command.

This displays the raw contents of the newly created public key asset on screen so that you can highlight and copy it.

Add a public key to the GitHub account.

Navigating to your personal [GitHub account SSH settings](#) and pasting this public string authorizes your Azure ML workspace compute to interact with your repositories.

From the terminal window in the Azure Machine Learning interface, run the git clone command.

With authenticating handshake materials successfully configured on both ends, you can now run the terminal cloning command securely without username or password prompts.

Incorrect Options

Add a private key to the GitHub account: You must never upload your private key (id_rsa) to any external platform, repository, or shared portal, as it compromises your machine's local identity and security credentials.

Question: 22

Exam Author

You manage an Azure Machine Learning workspace. You have an environment for training jobs which uses an existing Docker image. A new version of the Docker image is available. You need to use the latest version of the Docker image for the environment configuration by using the Azure Machine Learning SDK v2. What should you do?

- A. Change the description parameter of the environment configuration.
- B. Modify the conda_file to specify the new version of the Docker image.
- C. Use the create_or_update method to change the tag of the image.
- D. Use the Environment class to create a new version of the environment.

Answer: C

Explanation:

Technical Justification for Correct Answer (C)

Why C is the best option: To update the Docker image version in an Azure Machine Learning workspace using the Azure Machine Learning SDK v2, the most direct and appropriate method is to **use the create_or_update method to change the tag of the image**. This approach:

Targets the Specific Change: Directly updates the Docker image tag, which is the precise change required.

Utilizes Idempotency: The create_or_update method is idempotent, meaning it safely updates the existing environment if the image tag is the only change, or creates it if it didn't exist, ensuring consistency without unnecessary recreations.

Minimizes Disruption: This method minimizes potential disruptions by only updating the specified parameter (the Docker image tag) within the existing environment configuration.

Why Other Options are Less Suitable:

A. Change the description parameter of the environment configuration:

Irrelevant to the Task: Updating the description does not change the Docker image version used by the environment.

No Impact on Functionality: This change would not achieve the goal of using the latest Docker image version.

B. Modify the conda_file to specify the new version of the Docker image:

Incorrect Configuration Target: The conda_file is used for specifying Python package dependencies, not for updating the base Docker image.

Potential for Misconfiguration: Attempting this could lead to environment configuration errors.

D. Use the Environment class to create a new version of the environment:

Unnecessary Overhead: Creating a new environment version is more complex and unnecessary when the sole change is the Docker image tag.

Additional Management Burden: This approach would leave the old environment configuration intact, potentially leading to version management complexities.

Question: 23

Exam Author

You manage an Azure Machine Learning workspace.

You need to define an environment from a Docker image by using the Azure Machine Learning Python SDK v2.

Which parameter should you use?

- A. conda_file
- B. properties
- C. build
- D. image

Answer: D

Explanation:

Technical Justification for Selecting Option D

When utilizing the Azure Machine Learning Python SDK v2 to define an environment from a Docker image, the correct parameter to use is **D. image**. Here's why each option is either suitable or not:

D. image: This is the correct parameter for specifying a Docker image from which to define an environment in Azure Machine Learning using the Python SDK v2. The image parameter directly points to the Docker image, allowing Azure ML to pull the image and configure the environment accordingly. This approach is efficient for leveraging pre-built Docker images tailored for specific ML workload requirements.

A. conda_file: This parameter is used when the goal is to define an environment based on a Conda environment file (.yml). Since the question specifies defining the environment from a Docker image, conda_file is not applicable here. Conda files are used for managing Python packages and dependencies in a Conda environment, not for Docker image-based deployments.

B. properties: While properties can be used to customize various aspects of an Azure Machine Learning environment, it is too broad and not directly related to specifying the base image for the environment. Properties might include settings like environment variables or dependencies but not the foundational Docker image itself.

C. build: The build parameter or method is typically associated with building a Docker image from a Dockerfile or similar construct, not with defining an environment from an already existing Docker image. If you were creating a custom image from scratch (which is not the case here), build might be relevant, but for using an existing image, it's not the right choice.

Why D is the Best Choice:

Direct Relevance: image directly addresses the requirement of specifying a pre-existing Docker image.

Efficiency: Leveraging a pre-built Docker image streamlines the environment setup process.

Accuracy: Using image ensures that the environment is configured exactly as intended by the image's creators, minimizing configuration errors.

References

For further study and validation:

Azure Machine Learning Python SDK v2 Documentation: <https://docs.microsoft.com/en-us/python/api/overview/azure/ml/?view=azure-ml-py-v2>

Environment Configuration in Azure Machine Learning: <https://docs.microsoft.com/en-us/azure/machine-learning/how-to-configure-environment>

Question: 24

Exam Author

A company has multiple data science teams working on separate machine learning projects. The company requires models to be auditable, reusable, and governed centrally across teams. The models must allow team-level isolation for billing. You need to establish the foundation for governed machine learning operations. Which action should you perform first?

- A. Register shared datasets in a central storage account.
- B. Create a shared hub workspace and project workspaces for each team.
- C. Create a resource group for shared machine learning assets.
- D. Create a shared Azure Machine Learning workspace.

Answer: B

Explanation:

Technical Justification for Correct Answer: B

Establishing the foundation for governed machine learning operations, with requirements for auditable, reusable, centrally governed models, team-level isolation for billing, and support for multiple data science teams, necessitates a structured approach. Here's why creating a shared hub workspace and project workspaces for each team (Option B) is the optimal first step, and why other options are less suitable as the initial action:

Why B is the Correct First Action:

Central Governance & Team Isolation: Creating a shared hub workspace (likely leveraging Azure DevOps or similar for governance) with project workspaces for each team lays the groundwork for central governance. This setup allows for defined access controls, enabling team-level isolation while maintaining central oversight.

Billing Isolation: Project workspaces can be configured to isolate billing at the team level, aligning with the company's requirement for team-level billing management.

Foundation for Future Steps: This structure provides a foundation for subsequently registering shared datasets (A), creating resource groups for shared assets (C), or utilizing a shared Azure Machine Learning workspace (D) in a more organized, governed manner.

Why Other Options are Less Suitable as the First Action:

A. Register Shared Datasets in a Central Storage Account:

Lack of Governance Framework: Without a governing structure in place first, dataset registration lacks the oversight and access control framework needed for multi-team governance.

Premature Focus on Data Storage: Governance and team structure should precede detailed data management steps.

C. Create a Resource Group for Shared Machine Learning Assets:

Infrastructure Before Governance: While necessary, creating a resource group for shared assets is more of an infrastructure step that benefits from being guided by an already established governance and workspace structure.

Lacks Team Isolation Mechanism: On its own, a resource group does not inherently provide the team-level isolation required.

D. Create a Shared Azure Machine Learning Workspace:

Overly Tool-Centric First Step: While crucial for ML operations, creating a shared Azure Machine Learning workspace as the first step might not address the broader governance and team isolation requirements as effectively as establishing a hub/project workspace structure.

Assumes Unified Team Needs: A shared workspace might not initially accommodate the need for team-level isolation and separate project management as well as creating project workspaces within a hub.

Conclusion

Given the requirements, establishing a structured governance and team management framework is paramount. **Option B (Create a shared hub workspace and project workspaces for each team)** is the most appropriate first action as it addresses central governance, team isolation for billing, and provides a foundational structure for all subsequent operational and technical steps.

References

[1] Microsoft Learn - Design a governance and management structure for Azure Machine Learning] (<https://learn.microsoft.com/en-us/learn/modules/design-governance-structure-azure-ml/>)

[2] Azure Architecture Center - DevOps for Machine Learning](<https://azure.microsoft.com/en-us/solutions/devops-for-ml/>)

Question: 25

Exam Author

A team provisions an Azure Machine Learning environment by triggering pull requests. Deployments must be automated, auditable, and require approval before running. You need to select a deployment automation tool. Which tool should you use?

- A. Azure Monitor
- B. GitHub Actions
- C. MLflow
- D. Azure Machine Learning pipelines

Answer: B

Explanation:

Technical Justification

Selected Answer: B. GitHub Actions

GitHub Actions is the most suitable deployment automation tool for this scenario due to the following reasons:

Automated Deployments: GitHub Actions can be triggered by pull requests, automating the deployment process as required. Workflows can be defined to execute Azure Machine Learning deployment scripts.

Auditable: Each workflow run in GitHub Actions provides a detailed, logged history, making deployments auditable. This includes who triggered the run, the execution environment, and the outcome.

Approval Requirement: GitHub Actions supports the use of **Approval Actions** within workflows. This allows for a manual approval step to be inserted before the actual deployment to Azure Machine Learning, ensuring that deployments require approval before execution.

Why Other Options are Less Suitable:

A. Azure Monitor: Primarily a monitoring and analytics service, not designed for automating deployments based on pull requests or incorporating approval workflows directly. Its focus is more on alerting and troubleshooting rather than deployment automation.

C. MLflow: While MLflow is a comprehensive platform for managing the machine learning lifecycle and can integrate with deployment pipelines, it does not natively support triggering deployments from pull requests on GitHub with built-in approval steps as seamlessly as GitHub Actions. MLflow's strength lies in model management and experiment tracking.

D. Azure Machine Learning Pipelines: Although designed for automating ML workflows, Azure Machine Learning Pipelines are more focused on the workflow of machine learning tasks (e.g., data preparation, training) rather than the deployment process to production environments triggered by pull requests with approval gates. They are ideal for orchestrating ML tasks but less suited for the described deployment automation scenario.

References

For deeper insights into the selected tool and its integration capabilities:

1. **GitHub Actions Documentation - Automating Deployment**
<https://docs.github.com/en/actions/deployment/deploying-to-your-application>
2. **Azure Machine Learning with GitHub Actions**
<https://learn.microsoft.com/en-us/azure/machine-learning/how-to-deploy-github-actions>

Question: 26

Exam Author

A team develops and manages a conversational assistant by using Microsoft Foundry. The team requires generative AI to automatically evaluate every pull request of an agentic application and fail the build if safety thresholds are exceeded. You need to automate evaluations as part of CI. What should you configure?

- A. Blocklist applied to the model endpoint
- B. Content filter configured in warning mode
- C. Retrieval chunking strategy
- D. GitHub Actions workflow that executes the runs

Answer: D

Explanation:

Technical Justification for Correct Answer: D

To automate evaluations of every pull request for an agentic application using generative AI, integrated with Microsoft Foundry, and to fail the build if safety thresholds are exceeded as part of Continuous Integration (CI), the most suitable configuration is **D. GitHub Actions workflow that executes the runs**. Here's why:

Why D is the Correct Choice

Integration with CI Pipeline: GitHub Actions is a native CI/CD tool for GitHub, seamlessly integrating with the repository's pull request workflow. Configuring a GitHub Actions workflow allows for automated execution of generative AI evaluations directly within the CI pipeline, ensuring builds fail if safety thresholds are exceeded.

Automated Execution on Pull Requests: GitHub Actions can be triggered automatically on pull request events, aligning with the requirement to evaluate every pull request.

Execution of Custom/Complex Logic: A GitHub Actions workflow can execute scripts or APIs (e.g., Microsoft Foundry's APIs for generative AI evaluations), enabling the team to implement logic that checks safety thresholds and fails the build accordingly.

Why Other Options are Less Suitable

A. Blocklist applied to the model endpoint:

Purpose: Blocklist is primarily for preventing certain inputs or outputs, not for automating evaluations within a CI pipeline.

Limitation: Does not inherently integrate with CI/CD pipelines to fail builds based on evaluation results.

B. Content filter configured in warning mode:

Functionality: Alerts on violations but does not automatically fail a build in a CI context.

Shortcoming: Warning mode implies notification rather than enforcement, contradicting the requirement to fail the build.

C. Retrieval chunking strategy:

Relevance: Pertains to optimizing data retrieval or processing, not to automating CI/CD pipeline tasks or evaluating safety thresholds.

Misalignment: Does not address the automation of build failure based on generative AI evaluations.

Question: 27

Exam Author

Case Study -

This is a case study. Case studies are not timed separately from other exam sections. You can use as much exam time as you would like to complete each case study. However, there might be additional case studies or other exam sections. Manage your time to ensure that you can complete all the exam sections in the time provided. Pay attention to the Exam Progress at the top of the screen so you have sufficient time to complete any exam sections that follow this case study.

To answer the case study questions, you will need to reference information that is provided in the case. Case studies and associated questions might contain exhibits or other resources that provide more information about the scenario described in the case. Information provided in an individual question does not apply to the other questions in the case study.

A Review Screen will appear at the end of this case study. From the Review Screen, you can review and change your answers before you move to the next exam section. After you leave this case study, you will NOT be able to return to it.

To start the case study -

To display the first question in this case study, select the "Next" button. To the left of the question, a menu provides links to information such as business requirements, the existing environment, and problem statements. Please read through all this information before answering any questions. When you are ready to answer a question, select the "Question" button to return to the question.

Background -

Fabrikam Inc. is a mid-sized healthcare analytics company that provides population health dashboards and predictive insights to regional hospital systems across the United States. Fabrikam Inc. customers rely on near real time analytics to monitor patient flow, staffing needs, and readmission risks. They use multiple traditional

forecasting machine learning models for predictions.

Fabrikam Inc. has an established Microsoft Azure footprint. The company uses Jupyter Notebooks that run on a local server as the primary development environment. The data science team is experiencing scalability, asset management and code management issues with the current development platform. Fabrikam Inc. plans to migrate to a cloud-based development environment to mitigate the issues.

Additionally, the company plans to implement a Retrieval-Augmented Generation (RAG)-based chat application for client support. Leadership requires the application to be developed and deployed with a low operational risk.

Current Environment -

Fabrikam Inc. operates a single Azure subscription that has the following components:

Azure Data Lake Storage Gen2 that contains de-identified clinical and operational datasets

Azure AI Search indexing curated analytical documents and reference materials

A small set of Python-based training scripts maintained by data scientists

Azure OpenAI Service with deployed foundational models

A Microsoft Foundry resource for building a RAG-based solution

Evaluation data has manually defined expected responses.

The current challenges faced by the data science team include the following:

Model training jobs are run manually from notebooks.

Experiment tracking is inconsistent

Model versions are registered without standardized metadata.

Deployment is performed manually by data scientists, with limited rollback capability.

The team has no standardized evaluation process for generative AI outputs.

The environment currently allows public network access. Authentication relies on user accounts rather than managed identities. Compute targets are manually created and shared across experiments. This has led to resource contention during peak usage.

Business Requirements -

Fabrikam Inc. has the following business requirements for the modernization initiative:

Provide a conversational interface that answers analytics questions by using internal documents and datasets.

Ensure that sensitive healthcare-related data is not exposed outside the Fabrikam Inc. Azure tenant.

Enable repeatable and auditable model training and deployment processes.

Support experimentation to compare prompt strategies and fine-tuned models.

Align the model with the ranked preferences and optimize behavior for the long term.

Minimize disruption to existing analytics workloads during rollout.

Technical Requirements -

To support the business goals, Fabrikam Inc. identifies these technical requirements:

Use Azure Machine Learning workspaces to centrally manage data assets, models, and environments.

Implement experiment tracking and model versioning for all training jobs.

Orchestrate training and evaluation by using pipelines rather than manually running notebooks.

Deploy traditional machine learning models with support for staged rollout and rollback.

Improve RAG-based solution output quality.

Use the existing evaluation datasets that are based on real data with input-output pairs.

Apply advanced fine-tuning techniques only when prompt engineering is insufficient

Issues and Constraints -

Fabrikam Inc. must comply with internal security policies that require the company to restrict network access and avoid long-lived secrets. The data science team has limited Azure DevOps experience, so solutions must favor managed services and automation over custom infrastructure.

Cost predictability is important. Leadership prefers serverless or managed compute options where possible but is willing to approve dedicated compute for stable production workloads.

Problem Statement -

Fabrikam Inc. must design and implement an Azure-based AI operations solution that enables reliable training, evaluation, deployment, and iteration of generative AI models. The solution must support experimentation and gradual rollout while ensuring governance, security, and operational stability. The data science and platform teams must collaborate to deliver this solution by using Azure Machine Learning and Microsoft Foundry capabilities.

You need to recommend an experiment-tracking strategy that ensures consistent experiment results.

What should you recommend?

- A. Azure Machine Learning job output logs
- B. MLflow experiment tracking
- C. Application Insights logs
- D. Azure Monitor alerts

Answer: B

Explanation:

Technical Justification for Recommended Experiment-Tracking Strategy

Recommended Option: B. MLflow experiment tracking

Why B is the best choice:

Integration with Azure Machine Learning (AML): MLflow is natively integrated with Azure Machine Learning, making it seamless to track experiments across the platform. Fabrikam Inc. is adopting for their modernization initiative. This integration ensures consistent tracking of model training jobs, which is crucial given the manual and inconsistent tracking they currently face.

Comprehensive Experiment Tracking: MLflow provides detailed tracking of experiments, including hyperparameters, models, and metrics, facilitating repeatable and auditable model training processes as required by Fabrikam's business requirements.

Support for Multiple Data Science Tools: Since Fabrikam's data science team uses Python-based scripts and Jupyter Notebooks, MLflow's compatibility with these tools ensures a unified tracking experience across their workflow.

Alignment with Governance and Security: By centrally managing experiment tracking through MLflow within AML, Fabrikam can better enforce security policies and reduce operational risks, especially considering their need to restrict network access and avoid long-lived secrets.

Why Other Options are Less Suitable:

A. Azure Machine Learning job output logs

Limited Scope: While useful for job-specific logging, these logs do not provide the comprehensive experiment tracking capabilities (e.g., hyperparameter tuning, model versioning across experiments) that MLflow offers.

Lack of Integrated Experiment Comparison: Not designed for comparing experiments at scale, which is crucial for Fabrikam's experimentation needs.

C. Application Insights logs

Primarily for Application Telemetry: Best suited for monitoring application performance and usage, not tailored for deep machine learning experiment tracking.

Insufficient for Model Development Lifecycle: Lacks the specific features needed for tracking model training, hyperparameters, and versioning in the context of ML development.

D. Azure Monitor alerts

Alerting and Monitoring Focus: Designed for alerting on operational metrics and logs, not for tracking the nuances of machine learning experiments.

No Experiment Comparison Capabilities: Does not support the comparison of experiment runs, which is essential for optimizing model performance.

Conclusion: Given Fabrikam Inc.'s requirements for consistent, auditable, and governance-compliant experiment tracking integrated with their Azure Machine Learning workflow, **MLflow experiment tracking (B)** is the most appropriate choice. It addresses the scalability, asset management, and code management issues by providing a unified, managed platform for tracking experiments, thereby streamlining their machine learning development process.

References:

1. **Azure Machine Learning with MLflow Integration**
<https://docs.microsoft.com/en-us/azure/machine-learning/how-to-use-mlflow>
2. **MLflow Documentation for Experiment Tracking**

Question: 28

Exam Author

HOTSPOT -

A machine learning model is deployed to production in Azure Machine Learning and is actively serving predictions for a business application. The model was trained by using a historical dataset that represented expected input patterns at the time of deployment.

The team working on the model must ensure the following:

Changes in input data distribution are detected.

Appropriate actions are triggered when predefined thresholds are exceeded.

You need to configure monitoring to meet the requirements.

Which configuration should you use for each requirement? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Configure monitoring

Requirement

Detect changes in input data distribution.

Configuration

Increase compute size.
Enable data drift monitoring.
Retrain the model manually.

Trigger action when thresholds are exceeded.

Configure alerts.
Delete the endpoint.
Re-register the model.

Answer:

Configure monitoring

Requirement

Detect changes in input data distribution.

Configuration

Increase compute size.
Enable data drift monitoring.
Retrain the model manually.

Trigger action when thresholds are exceeded.

Configure alerts.
Delete the endpoint.
Re-register the model.

Explanation:

Detect changes in input data distribution → Enable data drift monitoring.

Data drift is the change in model input data over time that leads to model performance degradation. Enabling data drift monitoring compares your real-time inference production data against your baseline training data to measure statistical shifts in feature distributions.

Trigger action when thresholds are exceeded → **Configure alerts.**

When data drift metrics exceed predefined safe thresholds, configuring alerts (via Azure Monitor, email notifications, or webhooks) automatically triggers immediate notifications or secondary remediation pipelines.

Incorrect Answer

Increase compute size: This resolves hardware performance issues (like high CPU/memory usage or latency spikes), but does not analyze the underlying statistical characteristics of incoming data.

Retrain the model manually: This is an operational remediation action performed after drift has been detected and analyzed; it is not a structural monitoring setting.

Delete the endpoint: An extreme, destructive operational action that immediately disrupts application service availability rather than managing monitoring health gracefully.

Re-register the model: This tracks a new version asset in the workspace registry, but does not provide active event triggering mechanisms for live inference workloads.

Question: 29

Exam Author

You create a binary classification model. You use the Fairlearn package to assess model fairness. You must eliminate the need to retrain the model. You need to implement the Fairlearn package. Which algorithm should you use?

- A. `fairlearn.reductions.ExponentiatedGradient`
- B. `fairlearn.preprocessing.CorrelationRemover`
- C. `fairlearn.reductions.GridSearch`
- D. `fairlearn.postprocessing.ThresholdOptimizer`

Answer: D

Explanation:

Technical Justification for Correct Answer D

Scenario Requirements Recap:

Binary Classification Model already exists (no retraining needed)
Fairlearn package for model fairness assessment and implementation
Eliminate the need for retraining the model

Why D (`fairlearn.postprocessing.ThresholdOptimizer`) is the Best Choice:

No Retraining Requirement: Since the model is already trained and retraining is not an option, **post-processing techniques** are the only viable approach among the given options.

`fairlearn.postprocessing.ThresholdOptimizer` adjusts the decision threshold for different groups to achieve fairness, without altering the model itself.

Fairness Implementation: It directly addresses model fairness by optimizing thresholds for balanced accuracy across sensitive groups, aligning with the requirement to implement Fairlearn for fairness assessment and improvement.

Why Other Options are Less Suitable:

A. `fairlearn.reductions.ExponentiatedGradient`:

Requires Model Retraining: This reduction-based algorithm involves training a new model with fairness constraints, contradicting the "no retraining" requirement.

Focus: More on training fair models from scratch rather than post-hoc fairness adjustment.

B. `fairlearn.preprocessing.CorrelationRemover`:

Preprocessing Technique: Intended to remove correlations with sensitive attributes before training a model. Since the model is already trained, this approach is not applicable.

Requires Retraining: Effective use would necessitate retraining the model with the preprocessed data, which is not allowed.

C. `fairlearn.reductions.GridSearch`:

Model Selection and Retraining: Implies searching for the best model (possibly with different hyperparameters or fairness constraints) and retraining, which again, violates the no retraining condition.

Focus: On finding an optimal model through a grid search, not on post-hoc fairness adjustment.

References

[Fairlearn Documentation - ThresholdOptimizer](#)

[Fairlearn Overview for Context](#)

Question: 30

Exam Author

DRAG DROP -

A real-time endpoint is deployed in Azure Machine Learning to serve predictions to a web application.

Users report intermittent failures and unexpected responses when calling the endpoint.

You need to identify the appropriate troubleshooting action for each reported issue.

Which troubleshooting action should you perform for each issue? To answer, move the appropriate troubleshooting actions to the correct issues. You may use each troubleshooting action once, more than once, or not at all. You may need to move the split bar between panes or scroll to view content.

NOTE: Each correct selection is worth one point.

Troubleshooting actions

- Adjust autoscaling configuration.
- Restart the endpoint.
- Review deployment logs.
- Validate scoring script and model version.

Troubleshoot endpoint issues

Issue

- Requests fail after deployment updates.
- Endpoint responds but returns incorrect results.
- Endpoint times out under increased traffic.

Troubleshooting action

-
-
-

Answer:

Troubleshooting actions

Adjust autoscaling configuration.

Restart the endpoint.

Review deployment logs.

Validate scoring script and model version.

Troubleshoot endpoint issues

Issue

Requests fail after deployment updates.

Endpoint responds but returns incorrect results.

Endpoint times out under increased traffic.

Troubleshooting action

Review deployment logs.

Validate scoring script and model version.

Adjust autoscaling configuration.

Explanation:

Requests fail after deployment updates → Review deployment logs.

Why: When an endpoint immediately crashes or fails to serve requests right after an update, it typically signifies initialization faults. This includes web server startup failures, unhandled exceptions during container boot, or missing code packages. Inspecting the container's deployment logs provides the explicit error stack trace to isolate the failure.

Endpoint responds but returns incorrect results → Validate scoring script and model version.

Why: If the endpoint successfully completes the network handshake and returns an HTTP 200 response but delivers bad data, the infrastructure itself is perfectly healthy. The flaw is logical. It means either the wrong model iteration was packaged into the production registry, or a data schema/parsing bug was introduced into the `init()` or `run()` entry functions of the custom scoring script.

Endpoint times out under increased traffic → Adjust autoscaling configuration.

Why: Request timeouts under heavy load mean the allocated backend virtual machines cannot keep pace with concurrent traffic spikes. Modifying the autoscaling rules — such as lowering the target metric threshold or increasing the maximum instance limit — allows the system to provision resources dynamically before queues overflow and timeout limits are reached.

Incorrect Answer

Restart the endpoint:

Rebooting acts purely as a temporary operational band-aid. It can clear ephemeral container state or transient memory leaks, but it will completely fail to resolve permanent structural bugs like runtime code crashes, logic flaws in model versions, or resource starvation under real concurrent user stress.

Question: 31

Exam Author

A team manages an Azure Machine Learning workspace and deploys a model to an endpoint.

A deployed online endpoint shows inconsistent response times during periods of high traffic.

You need to identify potential performance degradation.

Which three metrics should you monitor? Each correct answer presents part of the solution. Choose three.

NOTE: Each correct selection is worth one point.

- A. Feature count
- B. Requests per minute
- C. Connections active
- D. Dataset size

Answer: BCE

Explanation:

Technical Justification for Correct Option (BCE)

To identify potential performance degradation causing inconsistent response times in a deployed Azure Machine Learning online endpoint during high traffic periods, monitoring the following three metrics is crucial:

B. Requests per Minute (RPM): This metric indicates the load on the endpoint. High RPM during the periods of inconsistency can point to scalability issues or inadequate resource allocation (e.g., insufficient vCPUs or memory), leading to slower response times. Monitoring RPM helps in determining if the endpoint is overwhelmed.

C. Connections Active: Active connections metrics reveal if there's a bottleneck in handling concurrent requests. A high number of active connections without a corresponding increase in RPM might indicate that requests are being queued or are waiting for a response, contributing to latency. This could suggest issues with the endpoint's configuration or the model's efficiency.

E. Request Latency: Directly measuring the time taken to process requests, request latency is the most obvious metric for identifying performance degradation. High latency during peaks in traffic can highlight inefficiencies in model optimization, inadequate scaling, or network issues between the client and the Azure endpoint.

Why Other Options are Less Suitable for This Scenario:

A. Feature Count: While important for model performance and data preprocessing, feature count does not directly correlate with real-time performance metrics like response time during high traffic. A high feature count might impact model inference time but would not fluctuate with traffic patterns.

D. Dataset Size: Relevant for training and deployment setup, dataset size affects model size and potentially inference time but does not dynamically change with traffic conditions. Thus, it's not indicative of performance degradation under high traffic specifically.

References

For deeper Dive into Azure Machine Learning Metrics and Monitoring:

Azure Monitor for Azure Machine Learning: <https://docs.microsoft.com/en-us/azure/machine-learning/how-to-integrate-azure-monitor>

Model Deployment and Management with Azure Machine Learning: <https://docs.microsoft.com/en-us/azure/machine-learning/how-to-deploy-and-where>

Question: 32

Exam Author

DRAG DROP -

A team deploys a machine learning model to a managed online endpoint. The team monitors model performance and data quality metrics in production.

When monitoring thresholds are exceeded, the team requires an automated operational response that notifies downstream systems.

You need to configure the monitoring solution to meet the requirements.

Which configuration should you associate with each requirement as a first step? To answer, move the appropriate configurations to the correct requirements. You may use each configuration once, more than once, or not at all. You may need to move the split bar between panes or scroll to view content.

NOTE: Each correct selection is worth one point.

Configurations

Enable autoscaling.

Trigger retraining pipeline.

Configure alert rules.

Configure a performance monitoring solution

Requirement

Automated response

Infrastructure optimization

Configuration

Answer:

Configurations

Enable autoscaling.

Trigger retraining pipeline.

Configure alert rules.

Configure a performance monitoring solution

Requirement

Automated response

Infrastructure optimization

Configuration

Configure alert rules.

Enable autoscaling.

Explanation:

Automated response → Configure alert rules.

Why: In Azure Monitor and Azure Machine Learning monitoring setups, an "Automated response" refers to triggering immediate systemic notifications or actions (like emails, webhooks, or automation runbooks) when performance metrics breach defined thresholds. Creating alert rules handles this automated response cycle directly.

Infrastructure optimization → Enable autoscaling.

Why: "Infrastructure optimization" is the practice of dynamically adjusting hardware resources to match actual workload demand. Enabling autoscaling ensures your compute targets scale up to handle traffic spikes smoothly and scale down during idle periods to prevent wasted cloud spending.

Incorrect Answer

Trigger retraining pipeline.

Why: While this is a common step in an MLOps lifecycle to fix data drift, it is a data model remediation action rather than a core metric performance monitoring solution configuration block. It is typically a downstream task launched by an alert rule or pipeline run, rather than a standalone option matching these specific architectural requirements.

Question: 33

Exam Author

HOTSPOT -

You use Azure Machine Learning to train models across multiple experiments by using the same workspace.

You must record training runs in a centralized location to compare results from different jobs.

During training, performance values must be captured so they appear in the experiment run history.

You need to configure experiment tracking.

What should you configure for each requirement? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Configure experiment tracking

Requirement

Ensure runs are recorded in the workspace experiment history.

Record performance values during model training.

Configuration

Data asset
MLflow tracking URI
Azure Monitor workspace

Prompt flow
Model registry
MLflow metrics logging

Answer:

Configure experiment tracking

Requirement

Ensure runs are recorded in the workspace experiment history.

Record performance values during model training.

Configuration

Data asset
MLflow tracking URI
Azure Monitor workspace

Prompt flow
Model registry
MLflow metrics logging

Explanation:

Ensure runs are recorded in the workspace experiment history → MLflow tracking URI

Why: In Azure Machine Learning, MLflow is the standard tool for experiment tracking. To ensure your local or remote training jobs are routed and recorded directly inside the workspace's centralized experiment tracking interface, you must configure the environment's MLflow tracking URI to point to your specific Azure ML workspace endpoint.

Record performance values during model training → MLflow metrics logging

Why: To capture dynamic training performance variables (such as loss, validation accuracy, or learning rates) continuously across training epochs, you use MLflow metrics logging functions (e.g., `mlflow.log_metric()`). This saves the scalar values sequentially, enabling visualization plots over time inside the experiment dashboard.

Incorrect Answer :

Data asset: A referenced asset pointer that handles links to data stores, raw datasets, or training files; it plays no role in tracking or routing code executions.

Azure Monitor workspace: A broad cloud monitoring tool utilized to collect infrastructure diagnostics, container platform telemetry, and web server health logs, rather than specific machine learning metrics.

Prompt flow: An integrated development framework built specifically to orchestrate, evaluate, and test Large Language Model (LLM) app workloads rather than log arbitrary telemetry for classic tabular model training.

Model registry: A specialized asset store designed exclusively to store, version, stage, and track finalized machine learning model binaries after training has fully concluded.